WellAbroad — System Architecture & Implementation Guide

AMD Developer Hackathon: ACT III · Team AFAM · Prepared 15 Oct 2026 · Audience: backend/architecture owner and teammates

1. Purpose of this document

This guide explains what WellAbroad does, how the system is structured, which Evolus and AMD pieces we use, and how each team member builds their part. It answers the open questions from planning: how Evolus fits, whether we need OpenRouter or an SDK, and how to run the model on Fireworks AI or vLLM on AMD.

Deadline note: the online phase on lablab.ai runs 12–17 October, followed by an on-site phase in Milan on 17–18 October. The working backend should be ready by **Friday 16 Oct night**, leaving Saturday for integration, demo and submission. (Monday 19 Oct is after the event.)

2. Product summary

WellAbroad helps tourists who face a language barrier at foreign clinics and hospitals.

Feature 1 — Document Explainer. The user provides a medical document by uploading a photo, taking a photo, or uploading a file (PDF or image). The system reads it and explains it in the user's own language. An AI assistant answers follow-up questions. Rule: **explain only, never invent.**

Feature 2 — Consultation Prep. The user chooses their comfortable language, then the country they are in and its local language. The system asks natural, adaptive symptom questions in the user's language (about 10, each skippable), then produces a summary in the local language for the doctor, plus a copy in the user's language to verify.

User flow: pick own language → pick country and local language → choose Feature 1 or Feature 2 → guided experience.

3. High-level architecture

```
[Next.js frontend]  (Vercel)
        |  HTTPS / JSON / SSE
        v
[FastAPI backend]   (our orchestration + business rules)
        |  REST + OpenAI-compatible chat
        v
[Evolus workspace]  extractor | agents | AI Tools | (workflow, libraries)
        |  OpenAI-compatible calls, using OUR model
        v
[Model endpoint]    Option A: vLLM on AMD Developer Cloud (MI300X)
                    Option B: Fireworks AI (managed)
```

Key idea: **the frontend never talks to Evolus or the model directly.** Only the FastAPI backend holds API keys, calls Evolus, and enforces our safety rules. Evolus calls our model; our backend does not need to.

4. Components and responsibilities

Component	Owner	Responsibility
Next.js frontend	Frontend dev	Language/country selection, file upload and camera capture, chat UI, question UI with Skip button and quick-reply chips, summary screen, print/PDF view
FastAPI backend	Backend lead	Session state, calls to Evolus, question budget logic, mandatory final questions, red-flag handling, JSON parsing and retries, caching, rate limiting
Evolus workspace	Backend lead (shared team access)	Extractor, agents, AI Tools, model connection, API keys
Model endpoint	Backend lead	vLLM on AMD or Fireworks; connected inside Evolus
Session store	Backend lead	In-memory dict or SQLite with TTL; nothing permanent

5. Evolus: what it is and how we use it

Evolus is an AI orchestration platform. For the hackathon it gives each team a free **event workspace** with three ways in:

Surface	Base URL	Auth	Use for
Agent chat (OpenAI-compatible)	`https://api.evolus.ai/v1`	`Authorization: Bearer <application API key>`	Talking to our agents
REST API	`https://api.evolus.ai/api/v1`	`X-ApiKey` + `X-Application-Id`	Extractors, AI Tools, workflows, libraries
MCP server	`https://api.evolus.ai/mcp`	`X-ApiKey` + `X-Application-Id`	Optional: use from MCP-capable AI clients

Workspace rules to remember

- Create the workspace **only from the official event link** (published on evolus.ai/en/hackathon). A workspace created earlier is not an event workspace.
- One workspace per team, up to 6 members. Create one and invite everyone.
- Limits: 50,000 platform credits and 60 API requests per minute, free until 15 Nov 2026.
- Calls to our own model do not consume platform credits. Document reading, AI Tools and library indexing do. When credits run out, those calls return `402 Payment Required`.
- The API keys page in the workspace shows the API key and the space ID (used as `X-Application-Id`).

6. Do we need OpenRouter or an SDK?

OpenRouter: no. Evolus connects directly to any OpenAI-compatible endpoint, so Fireworks or our own vLLM server is enough. OpenRouter would only add another layer, another key and another point of failure.

Fireworks SDK / vLLM SDK: no. We never call the model from our code in the main design. In Evolus we enter three values under the workspace's model settings: base URL, model name and API key. Evolus does the calling.

- vLLM is not an SDK. It is a server we run on an AMD GPU machine.
- Fireworks is a hosted API. We only need an API key.

SDK for our own backend: optional. To call Evolus from FastAPI we can use plain `httpx`, or the standard `openai` Python package pointed at `https://api.evolus.ai/v1` for agent chat. Both work. The extractor and AI Tools are plain REST calls, so `httpx` is enough for them.

(Based on the Evolus developer page. Confirm in the workspace's model settings when you add the model.)

7. Model options

Option A — vLLM on AMD Developer Cloud (preferred for the AMD story)

1. Get AMD Developer Cloud credits through the AMD AI Developer Program. **Approval takes 2–3 business days, so apply today** and also ask on the lablab Discord whether credits were already granted.
2. Create a GPU droplet (AMD Instinct MI300X, 192 GB), using the vLLM quick-start image if offered.
3. Serve an open model with an OpenAI-compatible API:

```
vllm serve Qwen/Qwen2.5-7B-Instruct \
  --api-key $VLLM_KEY \
  --served-model-name wellabroad \
  --max-model-len 16384 \
  --gpu-memory-utilization 0.85 \
  --enable-prefix-caching \
  --enable-auto-tool-choice \
  --tool-call-parser hermes
```

Notes:

- `--api-key` is mandatory in practice: without it anyone who knows the address can use the endpoint.

- `--tool-call-parser hermes` fits Qwen models. Evolus agents use tools, so the model must support function calling.
- `--enable-prefix-caching` speeds up chat because each turn reuses the same document context.
- If the server crashes on ROCm, try adding `--enforce-eager` (other teams reported this fixed instability).
- A larger Qwen model can be tried later; the 192 GB memory allows it. Start small.
- Endpoint for Evolus: `http://<droplet-ip>:8000/v1`, model name = the served model name, key = the `--api-key` value.
- Test with curl before adding it to Evolus.
- Shut the instance down when not testing to save credits.

Option B — Fireworks AI (fast start and fallback)

- Base URL: `https://api.fireworks.ai/inference/v1`
- Model: `accounts/fireworks/models/<model>` (choose one that supports function calling and the languages we need)
- Key: Fireworks API key
- Evolus states that both routes accept AMD cloud credits.

Recommended sequence: connect Fireworks first so the team can build today, then switch to vLLM on AMD when credits arrive. In Evolus this is a settings change, not a code change.

Language caution: test every target language early (for example Urdu, Arabic, Hindi, plus the local language of the countries in the demo). Quality varies by model and language.

8. What to create inside Evolus

8.1 Model connection

Add the endpoint (Option A or B), test the connection, and select it in each agent.

8.2 Extractor — "Medical Document Reader"

Reads photos, scans and PDFs (with OCR) and returns JSON in a schema we define. It is the single entry point for all three Feature 1 inputs (upload photo, take photo, upload document), because to the backend they are all just a file. Document reading uses

platform credits, so cache results and test the cost per document early.

Suggested schema (adjust fields in the workspace):

```
{
  "document_type": "lab_report | prescription | discharge_summary |
doctor_note | imaging_report | invoice | other",
  "source_language": "string",
  "patient_info_present": true,
  "date": "string",
  "sections": [{"title": "string", "text": "string"}],
  "lab_results": [{"name": "string", "value": "string", "unit": "string",
"reference_range": "string", "flag": "string"}],
  "medications": [{"name": "string", "dose": "string", "frequency": "string",
"duration": "string", "instructions": "string"}],
  "diagnoses_or_findings": ["string"],
  "instructions_to_patient": ["string"],
  "follow_up": "string",
  "unreadable_parts": ["string"]
}
```

The extractor must **transcribe only**: copy what is written, mark unclear parts in `unreadable_parts`, never guess.

8.3 Agent 1 — "Document Explainer"

Explains the extracted document and answers follow-up questions in the user's language.

Instruction essentials:

- Use only the document content provided. If the answer is not in the document, say so and suggest asking the doctor or pharmacist.
- Point to the part of the document each explanation comes from (field name or short quote).
- Explain in simple language. Keep drug names, values and units exactly as written.
- Never diagnose. Never advise starting, stopping or changing a medication or dose.
- If the user describes an emergency (chest pain, difficulty breathing, severe bleeding, loss of consciousness, etc.), tell them to contact local emergency services immediately.
- Always answer in the language the user selected.

8.4 Agent 2 — "Consult Interviewer"

Generates the next adaptive question (see section 10). Replies with JSON only.

8.5 Agent 3 — "Consult Summarizer" (can be the same agent with a different mode)

Turns collected answers into a clean English summary for the doctor: complaint, onset, duration, severity, associated symptoms, medications, allergies, conditions, and which items were skipped. English acts as the **pivot language** so translation errors do not compound.

8.6 AI Tool — `translate`

Used to render the English summary into the country's local language (style: formal) and again into the user's language for verification. Called at `POST /api/v1/ai-tools/translate/run` with a JSON body `{ "text": "...", "options": { "target": "German", "style": "formal" } }`. It uses platform credits.

8.7 Optional extras

- **Workflow:** chain extractor → explainer agent into one call, started with `POST /api/v1/workflows/{id}/execute` and read by polling the execution. Only worth doing if time remains; direct calls from FastAPI are simpler.
- **Libraries:** knowledge bases for agents. Not needed, because documents are short enough to send in full.
- **Other AI Tools** (`simplify`, `summarize`) could be shown as extras but are not required.

9. Feature 1 — Document Explainer in detail

1. Frontend uploads the file (or camera capture) to `POST /docs`.
2. Backend validates type and size (Evolus documents up to 20 MB), then sends the file to the extractor: `POST /api/v1/agents/{agentId}/json/{extractorId}/ask-file` with `X-ApiKey` and `X-Application-Id`.
3. Backend stores the extracted JSON in the session (not the file) and caches it by file hash.
4. Backend starts a conversation with the Document Explainer: first message contains the user's language and the extracted JSON, asking for a structured explanation.

5. Backend returns the explanation. The response header `X-Conversation-Id` is stored in the session.
6. Follow-up questions go to `POST /docs/{id}/chat`. The backend sends only the new user message with the same `X-Conversation-Id`; Evolus loads the history. Use `stream: true` and forward the stream as SSE to the frontend.

Note: agent chat accepts **text only**, which is why images must go through the extractor first. Model settings like `temperature` come from the agent and cannot be set per request.

10. Feature 2 — Consultation Prep in detail

Principle: a fixed *frame* with LLM-chosen questions, not a fixed script.

Flow

1. Opening question (fixed): "What is bothering you most right now?" in the user's language.
2. After every answer the backend sends the Interviewer a state object:

```
{
  "user_language": "ur",
  "transcript": [{"q": "...", "a": "..."}],
  "collected": {"chief_complaint": "headache", "duration": "3 days"},
  "skipped": ["severity"],
  "questions_asked": 4,
  "max_questions": 10
}
```

3. The Interviewer replies with JSON only:

```
{
  "done": false,
  "red_flag": false,
  "target": "associated_symptoms",
  "question": "Do you also feel sick to your stomach or sensitive to light?",
  "quick_options": ["Yes", "No", "Not sure"]
}
```

4. The frontend shows the question with quick-reply chips, a free-text box and a **Skip** button. Skipped items are recorded as "not provided", never as "no", and are never asked again.
5. The loop ends when the Interviewer sets `done: true` or the budget (about 10) is reached.
6. Summary generation: Summarizer creates the English summary → `translate` to the local language (for the doctor) → `translate` to the user's language (for verification). The UI shows both side by side with a print/PDF option.

Interviewer instructions (essentials)

- Ask about what the person feels only. Never name a cause or diagnosis.
- Choose the next question from what is already known, using a standard symptom checklist: when it started, where, how severe, what it feels like, what makes it better or worse, associated symptoms. Skip anything irrelevant.
- One short, plain-language question per turn. Offer 2–4 quick options when sensible.
- Never repeat a question or ask about skipped items again.
- Set `red_flag: true` for emergency signs.
- Output valid JSON and nothing else.

Rules enforced in backend code (not left to the model)

- In the last three slots, always ask: current medications, allergies, known conditions (skippable).
- Hard cap on questions.
- Parse defensively: strip code fences, validate against a schema, retry once on invalid JSON, then fall back to a safe default question.
- If `red_flag` is true, show an urgent-care warning with a prompt to contact local emergency services, while still allowing the user to continue.

11. Backend API contract (frontend ↔ FastAPI)

Endpoint	Purpose	Returns
`POST /session`	Body: `ui_lang`, `country`, `local_lang`	`session_id`

Endpoint	Purpose	Returns
`POST /docs`	Multipart file (photo, camera or document)	`doc_id`, extracted status
`GET /docs/{id}/explanation`	Plain-language explanation	text in user language
`POST /docs/{id}/chat`	Follow-up question	SSE stream
`POST /consult/{sid}/start`	Begin interview	first question
`POST /consult/{sid}/answer`	Answer or `skip: true`	next question or `done`
`GET /consult/{sid}/summary`	Final summary	local-language text, user-language text, red-flag info

12. Environment variables

```
EVOLUS_API_KEY=
EVOLUS_APP_ID=
EVOLUS_EXTRACTOR_AGENT_ID=
EVOLUS_EXTRACTOR_ID=
EVOLUS_EXPLAINER_MODEL=<applicationId>:<agentId>
EVOLUS_INTERVIEWER_MODEL=<applicationId>:<agentId>
EVOLUS_SUMMARIZER_MODEL=<applicationId>:<agentId>
ALLOWED_ORIGINS=https://<frontend-domain>
# Model credentials live inside Evolus, not in our backend:
# Fireworks key OR vLLM URL + --api-key value
```

List the agent model IDs with `GET https://api.evolus.ai/v1/models`; each `id` is already in `applicationId:agentId` form.

13. Calls cheat sheet

```
# List agents
curl https://api.evolus.ai/v1/models -H "Authorization: Bearer
$EVOLUS_API_KEY"

# Chat with an agent
curl https://api.evolus.ai/v1/chat/completions \
  -H "Authorization: Bearer $EVOLUS_API_KEY" -H "Content-Type:
application/json" \
  -d '{"model":"<applicationId>:<agentId>","messages":
[{"role":"user","content":"Hello"}]}'

# Extract data from a document
curl -X POST
"https://api.evolus.ai/api/v1/agents/$AGENT_ID/json/$EXTRACTOR_ID/ask-file" \
  -H "X-ApiKey: $EVOLUS_API_KEY" -H "X-Application-Id: $EVOLUS_APP_ID" -F
"file=@report.pdf"

# Translate text
curl -X POST https://api.evolus.ai/api/v1/ai-tools/translate/run \
  -H "X-ApiKey: $EVOLUS_API_KEY" -H "X-Application-Id: $EVOLUS_APP_ID" -H
"Content-Type: application/json" \
  -d '{"text":"...","options":{"target":"English","style":"formal"}}'
```

If an extractor has a webhook configured, the call is queued; add `?useWebhook=false` to get the data in the response.

14. Safety, privacy and quality

- **No invention:** transcribe first, explain second, and require every explanation to trace back to the document. Optionally add a verification call: "Is every statement supported by the document? List unsupported ones," and remove those.
- **No diagnosis, no dosage advice:** repeated in agent instructions and visible in the UI as a short disclaimer.
- **Emergency handling:** keyword and model-based red flags lead to an urgent-care message.
- **Privacy:** do not store uploaded files. Keep extracted data in session memory with a time limit. Evolus states that AI Tools process files in memory without storing them. Mention this in the demo. Consider the `anonymize` AI Tool as an optional step for

removing personal data.

- **Keys:** keep all keys on the backend. Never expose them to the browser. Restrict CORS to our frontend domain.
- **Limits:** 60 requests per minute per workspace, so avoid unnecessary calls and cache extraction results.

15. Work split and schedule

When	Backend lead	Frontend dev	Others
Wed 14–Thu 15	Create Evolus workspace from event link; connect Fireworks; apply for AMD credits; build extractor schema and test with 3–4 real documents	Build screens and camera/upload with mock API	Collect sample documents in several languages; write test cases
Thu 15–Fri 16	Explainer agent, chat with streaming, Interviewer + Summarizer, translate calls; deploy backend	Connect real endpoints, Skip button, chips, summary screen	Test all target languages
Fri 16 (night)	Switch model to vLLM on AMD if credits are available; capture `rocm-smi` evidence and tokens/sec	Polish, error states, print/PDF	Prepare demo script
Sat 17–Sun 18	Freeze features, fix bugs, fallback ready	Final UI polish	Record demo, write submission, present

16. Risks and fallbacks

Risk	Mitigation
AMD credits arrive late	Start on Fireworks; switch later

Risk	Mitigation
Extractor credits drain quickly	Test cost per document now; cache by file hash; use a few demo documents
Weak output in a target language	Test early; pick another model; keep English pivot
Agent returns invalid JSON	Schema validation, one retry, safe fallback question
vLLM crashes on ROCm	Add `--enforce-eager`; keep Fireworks configured as backup
Endpoint down during judging	Keep the instance running; prepare recorded demo and cached sample outputs
Rate limit (60/min)	Cache, queue, avoid parallel bursts

17. Items to verify in the workspace

- Exact credit cost of one extractor call and one translate call.
- Which image and file types the extractor accepts, including phone photos.
- Whether the chosen Fireworks model supports function calling in Evolus.
- That JSON-only replies from the Interviewer are reliable with our model.
- Output quality for each language pair we will demo.

18. Submission and demo checklist

- Working end-to-end demo of both features with at least two language pairs.
- Evolus clearly used: extractor, agents and AI Tool shown in the architecture slide.
- AMD clearly used: model served by vLLM on MI300X with ROCm, with GPU metrics.
- Architecture diagram, short privacy and safety explanation, and a note that the app explains but never diagnoses.
- Live URL for the frontend, public repository, and demo video.